

10.3.3 动手实现教程：从零写出导航集成层

这份教程给谁看：已经上完 10.3.3 原理课（知道 AMCL、代价地图、DWB、行为树是干什么的），也会用 topic / service / action 写简单节点，但面对"自己实现一套 Nav2 导航"时不知道从哪里下手的你。

读完并做完后你能：不看现成代码，自己从零写出让 Nav2 跑起来的全部"胶水代码"，并且说清每一行为什么非写不可。

前置条件：项目位于 ~/mecamind_ros2，第三课环境已构建通过（scripts/test_cp3.sh 能跑通）。原理不再重复，需要时回看 docs/10.3.3_Nav2导航系统与路径规划.md。

做完之后：请接着读 docs/10.3.3_工程版源码走读.md ——拿你写的简化版对照仓库里的工程版逐文件串讲，补上本教程没覆盖的 waypoint_patrol、Nav2 参数文件和 CP3 验收链。

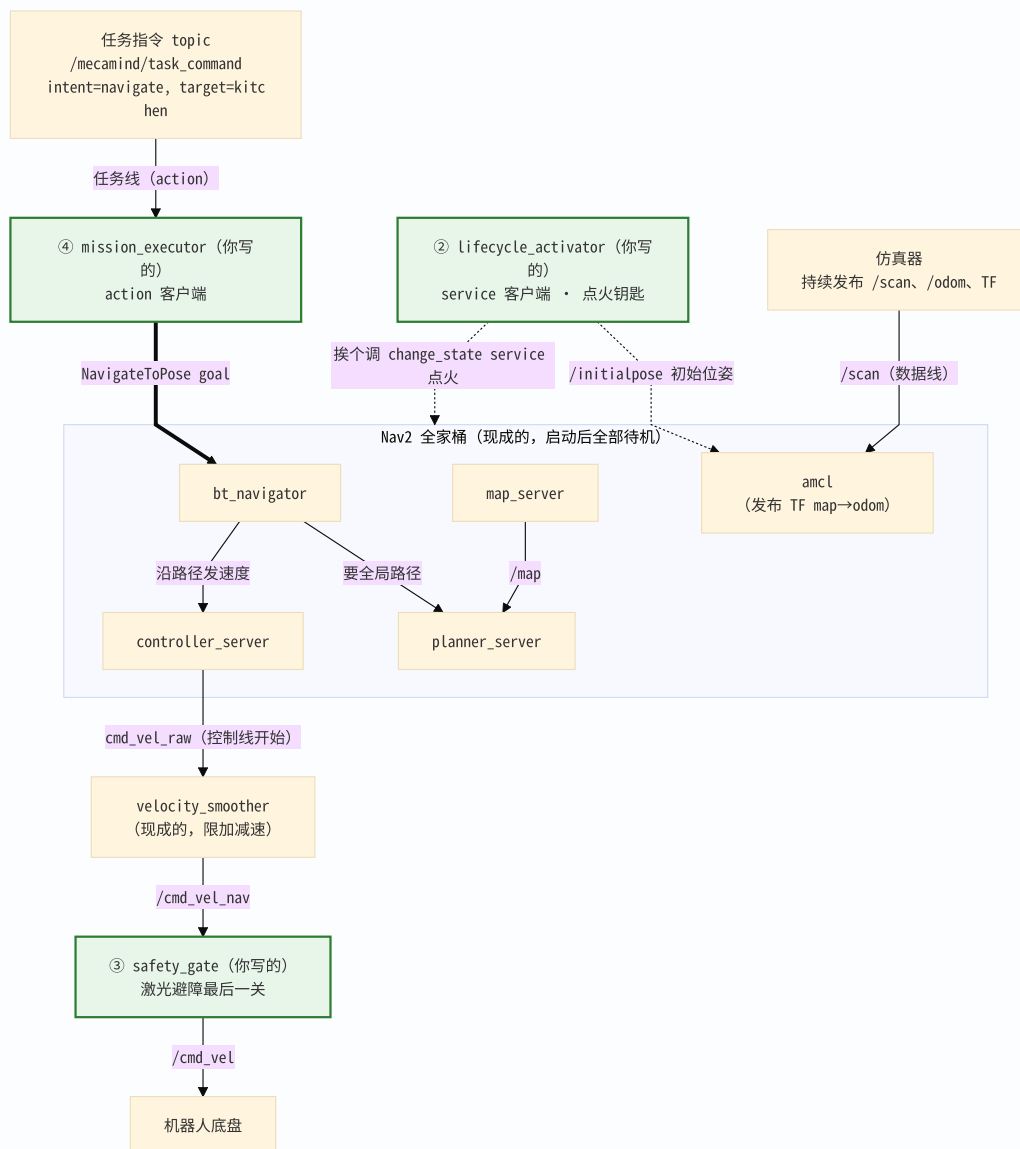
第 0 章 高屋建瓴：你要写的其实只有 4 个文件

先破除一个误解："实现 Nav2 导航"不等于实现 Nav2。AMCL、规划器、控制器这些算法节点都是安装好的现成程序（sudo apt install ros-jazzy-navigation2 就有了），你一行都不用写。你要写的是把它们拼成一个能用的系统的**集成层**，总共 4 个文件：

#	文件	用到的 ROS 2 机制	一句话职责
1	nav_utils.py	无（纯 Python）	工具箱：yaw 转四元数、读 YAML、拼 PoseStamped
2	lifecycle_activator.py	service 客户端	点火钥匙：把 9 个 Nav2 节点依次 configure → activate
3	safety_layer.py	topic 订阅 + 发布	安全门：Nav2 的速度指令过它这关才能到底盘
4	mission_executor.py	action 客户端	任务翻译官：把 "去厨房" 翻译成 NavigateToPose goal
5	navigation.launch.py	launch + remapping	总装图纸：把以上所有东西一条命令拉起来

（launch 文件算第 5 个，但它不是节点，是"图纸"。）

把整个系统压缩成一张图——**三条线，一把钥匙：**



(绿色的三个节点是你写的；虚线是“一把钥匙”——Nav2 节点启动后全部处于待机， lifecycle_activator 挨个调 service 点火，最后告诉 AMCL 初始位姿。)

对应你已学的三种通信机制：

- **任务线是 action**——导航耗时长、要反馈、可取消，正是 action 的用武之地；
- **控制线和数据线是 topic**——高频流式数据，发了就走；
- **点火是 service**——一问一答、要确认结果，正是 service 的用武之地。

后面每一章写一个文件。顺序刻意安排成**从易到难**：纯函数 → service → topic → action → launch。

第 1 章 不写一行代码，先把系统"手动开一遍"

写代码之前，先用命令行把每个待写文件的工作**手动做一遍**。做完你就知道代码要自动化的到底是什么——这比任何讲解都直接。

实验 1.1 手动"点火"一个 lifecycle 节点

单独启动一个 map_server（不启动其他任何东西）：

```
source ~/mecamind_ros2/install/setup.bash
ros2 run nav2_map_server map_server --ros-args \
  -p yaml_filename:=$HOME/mecamind_ros2/install/mecamind_tools/share/mecamind_tools/maps/mecamind_three_room_nav.y
aml
```

新开一个终端，验证它虽然启动了但没在工作：

```
ros2 topic echo /map --once --timeout 3 # 报错: topic does not appear to be published yet
ros2 lifecycle get /map_server         # unconfigured [1]
```

第一条命令会直接报错"话题还没被发布"——不是网络问题，是 map_server 此刻连 /map 的发布者都还没创建（发布者要到 configure 阶段才建）。这就是 lifecycle 节点的特性：启动 ≠ 工作。现在手动推它两把：

```
ros2 lifecycle set /map_server configure # 读地图文件、建发布者
ros2 lifecycle get /map_server           # inactive [2]
ros2 lifecycle set /map_server activate  # 真正开始发布
ros2 lifecycle get /map_server           # active [3]
ros2 topic echo /map --once --timeout 3 # 这回有地图了
```

再看一眼 `ros2 lifecycle set` 背后是什么：

```
ros2 service list | grep map_server
# /map_server/change_state ← set 就是在调这个 service
# /map_server/get_state    ← get 就是在调这个 service
```

结论：Nav2 有 9 个这样的节点，每启动一次都要手动敲 18 条命令、还得按依赖顺序敲。第 3 章要写的 `lifecycle_activator` 就是把这套手工活变成一个 service 客户端循环。（实验完 Ctrl+C 关掉 map_server。）

实验 1.2 手动给 Nav2 发一个导航目标 (action)

启动完整导航系统（用轻量后端，快）：

```
ros2 launch mecamind_bringup navigation.launch.py backend:=lite use_gui:=false with_rviz:=true
```

等 RViz 里出现地图和机器人后（约 15 秒），新终端直接用命令行当 action 客户端：

```
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose \
  "{pose: {header: {frame_id: map}, pose: {position: {x: -0.5, y: -1.2}, orientation: {w: 1.0}}}}" \
  --feedback
```

观察输出的三段式结构，这正是 action 协议的三个阶段：

1. `Goal accepted` ——服务端受理（对应代码里的 `goal response` 回调）；

2. 持续滚动的 `Feedback: distance_remaining ...` ——执行中反馈；
3. `Result + Status: SUCCEEDED` ——最终结果（对应 `result` 回调）。

结论：第 6 章要写的 `mission_executor`，核心就是用 Python 把这条命令自动化，外加一层"把 `kitchen` 查表变成 `(-0.5, -1.2)`"的翻译。

实验 1.3 看清速度链的三跳

系统保持运行，再发一次上面的导航目标，同时分三个终端观察：

```
ros2 topic echo /cmd_vel_raw --once    # 第 1 跳: Nav2 控制器的原始决策 (10Hz, 数值可能跳变)
ros2 topic echo /cmd_vel_nav --once     # 第 2 跳: velocity_smoother 平滑后 (加速度受限)
ros2 topic echo /cmd_vel --once         # 第 3 跳: safety_gate 放行后, 底盘真正执行的
```

再确认一件事——**机器人只订阅 `/cmd_vel`：**

```
ros2 topic info /cmd_vel    # 订阅者是仿真器 (底盘)
ros2 topic info /cmd_vel_raw # 发布者是 controller_server / behavior_server
```

结论：Nav2 的速度不是直接给底盘的，中间被"改了两次名字"。谁改的？launch 文件里的 remapping（第 7 章）。为什么要改？为了在链路中插进平滑器和安全门。第 5 章要写的 `safety_gate` 就是这条链的最后一环。

做完三个实验，回答三个问题再往下走（能答上来说明架构图真的在你脑子里了）：

1. 为什么 Nav2 节点启动后不直接工作，非要人点火？（提示：9 个节点有依赖顺序）
2. 导航用 action 而不是 service，最关键的两个理由是什么？
3. 如果把 `controller_server` 的输出直接映射到 `/cmd_vel`，系统会少什么能力？

第 2 章 搭好你自己的练习包

所有练习代码写进一个独立的新包 `nav_rebuild`，不碰现有工程——写坏了也不影响上课演示。

```
cd ~/mecamind_ros2/src
ros2 pkg create --build-type ament_python nav_rebuild \
  --dependencies rclpy geometry_msgs sensor_msgs std_msgs nav2_msgs \
  lifecycle_msgs action_msgs nav2_simple_commander
```

打开 `src/nav_rebuild/setup.py`，把 `entry_points` 改成（后面每写一个节点就多一行，现在先一次写全）：

```
entry_points={
    "console_scripts": [
        "my_activator = nav_rebuild.my_activator:main",
        "my_safety_gate = nav_rebuild.my_safety_gate:main",
        "my_executor_v1 = nav_rebuild.my_executor_v1:main",
        "my_executor_v2 = nav_rebuild.my_executor_v2:main",
    ],
},
```

以后每写完一个文件，构建 + 生效的固定动作是：

```
cd ~/mecamind_ros2
colcon build --packages-select nav_rebuild
source install/setup.bash
```

第 3 章 文件 1: nav_utils.py——先把"死知识"写成纯函数

为什么从它开始：后面三个节点都要"把 YAML 里的 {x, y, yaw} 变成 ROS 消息"。这种不依赖 ROS 运行时的逻辑抽成纯函数，是本项目贯穿始终的写法——纯函数不用启动任何节点就能测试。

新建 `src/nav_rebuild/nav_rebuild/nav_utils.py`：

```

import math

import yaml
from geometry_msgs.msg import PoseStamped, Quaternion

def quaternion_from_yaw(yaw: float) -> Quaternion:
    """平面机器人只绕 Z 轴转，轴角公式退化为  $z=\sin(yaw/2)$ ,  $w=\cos(yaw/2)$ 。"""
    q = Quaternion()
    q.z = math.sin(yaw * 0.5)
    q.w = math.cos(yaw * 0.5)
    return q

def load_yaml(path: str) -> dict:
    with open(path, "r", encoding="utf-8") as stream:
        data = yaml.safe_load(stream) or {}
    if not isinstance(data, dict):
        raise ValueError(f"YAML 顶层必须是字典: {path}")
    return data

def pose_stamped_from_dict(item: dict, stamp) -> PoseStamped:
    """{"x":..., "y":..., "yaw":...} -> PoseStamped (默认 map 坐标系)。"""
    pose = PoseStamped()
    pose.header.stamp = stamp
    pose.header.frame_id = str(item.get("frame_id", "map"))
    pose.pose.position.x = float(item["x"])
    pose.pose.position.y = float(item["y"])
    pose.pose.orientation = quaternion_from_yaw(float(item.get("yaw", 0.0)))
    return pose

```

三个理解要点，写的时候在心里过一遍：

1. **yaw** 为什么除以 2——四元数的定义如此（转角的一半进三角函数）。反向公式 $yaw = 2 * \text{atan2}(z, w)$ ，顺手记住。
2. 为什么必须是 **PoseStamped** 而不是 **Pose**——同样的坐标 (1, 2) 在 **map** 系和 **odom** 系是两个物理位置，不带 **frame_id** 的坐标在机器人系统里毫无意义。
3. **stamp** 为什么由调用方传——只有节点才有时钟（仿真里还得是 **sim time**），纯函数自己拿不到。

检验点（不需要构建，直接跑——注意要 **cd** 进包目录，**Python** 才找得到模块）：

```

source ~/mecamind_ros2/install/setup.bash
cd ~/mecamind_ros2/src/nav_rebuild
python3 -c "
from nav_rebuild.nav_utils import quaternion_from_yaw
import math
q = quaternion_from_yaw(math.pi / 2)
print(f'yaw=90° -> z={q.z:.3f}, w={q.w:.3f}') # 应为 z=0.707, w=0.707
"

```

对照工程版：src/mecamind_tools/mecamind_tools/nav_utils.py。多出来的东西：~ 路径展开、更完整的错误提示、initial_pose_from_dict 别名。都是易用性，不是原理。

第 4 章 文件 2: my_activator.py——用 service 客户端点火

目标：把实验 1.1 里手敲的 `ros2 lifecycle get/set` 变成代码。写完你将掌握 ROS 2 service 客户端的标准三步：`wait_for_service` → `call_async` → `spin_until_future_complete`。

写之前问自己三个问题：

1. 我要调用的 service 叫什么、类型是什么？ → `/<节点名>/get_state`（类型 `LifecycleMsgs/srv/GetState`）和 `/<节点名>/change_state`（类型 `ChangeState`）。
2. 节点当前可能处于哪些状态？我分别该做什么？ → `active`：不用管；`unconfigured`：先 `CONFIGURE` 再 `ACTIVATE`；`inactive`：只差 `ACTIVATE`。
3. 这个节点是常驻的还是一次性的？ → 一次性：干完活就退出，用进程返回码报告成败。所以主循环不用 `rclpy.spin(node)`。

新建 `src/nav_rebuild/nav_rebuild/my_activator.py`：

```

import rclpy
from lifecycle_msgs.msg import State, Transition
from lifecycle_msgs.srv import ChangeState, GetState
from rclpy.node import Node

class MyActivator(Node):
    def __init__(self) -> None:
        super().__init__("my_activator")
        self.declare_parameter("node_names", ["map_server"])
        self.declare_parameter("timeout_sec", 20.0)

    # ---- service 客户端标准三步，看仔细，后面一辈子都这么写 ----
    def get_state(self, node_name: str):
        client = self.create_client(GetState, f"/{node_name}/get_state")
        timeout = float(self.get_parameter("timeout_sec").value)
        if not client.wait_for_service(timeout_sec=timeout):          # 第 1 步：等服务端上线
            self.get_logger().error(f"{node_name}: get_state 服务不在线（节点没启动？）")
            return None
        future = client.call_async(GetState.Request())                # 第 2 步：异步发请求
        rclpy.spin_until_future_complete(self, future, timeout_sec=timeout) # 第 3 步：等响应
        if future.result() is None:
            return None
        state = future.result().current_state
        self.get_logger().info(f"{node_name} 当前状态: {state.label} [{state.id}]")
        return state.id

    def change_state(self, node_name: str, transition_id: int) -> bool:
        client = self.create_client(ChangeState, f"/{node_name}/change_state")
        timeout = float(self.get_parameter("timeout_sec").value)
        if not client.wait_for_service(timeout_sec=timeout):
            return False
        request = ChangeState.Request()
        request.transition.id = transition_id
        future = client.call_async(request)
        rclpy.spin_until_future_complete(self, future, timeout_sec=timeout)
        return future.result() is not None and future.result().success

    # ---- 核心逻辑：先查状态，再补齐缺的迁移（而不是盲目全来一遍） ----
    def bring_up(self, node_name: str) -> bool:
        state = self.get_state(node_name)
        if state is None:
            return False
        if state == State.PRIMARY_STATE_ACTIVE:
            return True
        if state == State.PRIMARY_STATE_UNCONFIGURED:
            if not self.change_state(node_name, Transition.TRANSITION_CONFIGURE):
                return False
            state = self.get_state(node_name) # configure 后重新确认
        if state == State.PRIMARY_STATE_INACTIVE:
            return self.change_state(node_name, Transition.TRANSITION_ACTIVATE)
        self.get_logger().error(f"{node_name} 处于意外状态 id={state}")
        return False

def main(args=None) -> None:

```



```

rclpy.init(args=args)
node = MyActivator()
names = [str(n) for n in node.get_parameter("node_names").value]
ok = True
for name in names:
    ok = node.bring_up(name) and ok # 失败不中断，一次跑完看到所有故障点
node.get_logger().info("全部激活成功" if ok else "有节点激活失败")
node.destroy_node()
rclpy.shutdown()
raise SystemExit(0 if ok else 1) # 返回码让脚本/launch 能判断成败

if __name__ == "__main__":
    main()

```

检验点：重演实验 1.1，但这次点火的是你的代码。

```

# 终端 1: 起一个"待机"的 map_server (命令同实验 1.1)
ros2 run nav2_map_server map_server --ros-args \
  -p yaml_filename:=$HOME/mecamind_ros2/install/mecamind_tools/share/mecamind_tools/maps/mecamind_three_room_nav.yaml

# 终端 2: 构建后用你的激活器点火
cd ~/mecamind_ros2 && colcon build --packages-select nav_rebuild && source install/setup.bash
ros2 run nav_rebuild my_activator
echo $? # 激活器的返回码，应为 0 ($? 只反映上一条命令，别隔着敲)
ros2 topic echo /map --once --timeout 3 # 有地图 = 成功

```

两个进阶思考（工程版多出来的部分，答案都在对照文件里）：

1. 激活 AMCL 之后它就能定位了吗？——不能，粒子滤波必须有人告诉它初值。工程版在 AMCL 激活后立刻向 `/initialpose` 发布初始位姿，而且 QoS 用了 `TRANSIENT_LOCAL`（发布者保留最后一条消息，晚到的订阅者也能补收），还连发 5 次防错过。想一想：如果用默认 QoS 只发一次，会发生什么？
2. 协方差为什么不填 0？——填 0 等于宣称"绝对确定在这"，粒子撒不开，激光匹配反而无法把小误差修掉。工程版给 x/y 各 0.25（标准差 0.5 米）、yaw 约 15°。

对照工程版： `src/mecamind_tools/mecamind_tools/lifecycle_activator.py`。重点看 `publish_initial_pose`（QoS + 协方差）和参数化的重试/延时逻辑。

第 5 章 文件 3: `my_safety_gate.py`——用 topic 写速度链最后一关

目标：写一个"中间人"节点：订阅上游速度指令和激光，前方太近就把前进分量清零，再转发给底盘。写完你将掌握本项目最重要的一个设计模式：**纯函数算逻辑，节点只做接线**。

写之前问自己三个问题：

1. 我订阅什么、发布什么？ → 订阅 `LaserScan`（环境）+ `Twist`（上游指令）；发布 `Twist`（放行后的指令）。
2. 数据由谁驱动？ → 由指令驱动：每来一条指令，用最新缓存的激光评估一次再转发。激光回调只缓存不发布——没有指令时机器人本来就不动，评估无意义。
3. 激光数据怎么从一维数组变成"前方距离"？ → `LaserScan` 的第 `i` 个测距值对应角度 `angle_min + i * angle_increment`，把角度落在前方 $\pm 0.45 \text{ rad}$ （约 $\pm 26^\circ$ ）扇区内的有效值取最小。

新建 `src/nav_rebuild/nav_rebuild/my_safety_gate.py`：

```

import math

import rclpy
from geometry_msgs.msg import Twist
from rclpy.node import Node
from sensor_msgs.msg import LaserScan

# ----- 纯函数部分：不碰 ROS，可以直接单元测试 -----

def front_min_distance(ranges, angle_min: float, angle_increment: float,
                      half_width: float = 0.45) -> float:
    """前方扇区 (0 弧度  $\pm$  half_width) 内最近的有效障碍距离，无障碍返回 inf。"""
    best = float("inf")
    for index, value in enumerate(ranges):
        if not math.isfinite(value) or value < 0.02: # inf/NaN 是"没打到"，太近是自身外壳
            continue
        angle = angle_min + index * angle_increment
        angle = math.atan2(math.sin(angle), math.cos(angle)) # 归一化到 (-pi, pi]
        if abs(angle) <= half_width:
            best = min(best, value)
    return best

def gate_forward(command: Twist, front_dist: float, stop_dist: float) -> Twist:
    """前方障碍进入 stop_dist 内则清零前进分量；倒车和旋转永远放行（脱困需要）。"""
    gated = Twist()
    gated.linear.x = command.linear.x
    gated.linear.y = command.linear.y
    gated.angular.z = command.angular.z
    if command.linear.x > 0.0 and front_dist < stop_dist:
        gated.linear.x = 0.0
    return gated

# ----- 节点部分：只做接线 -----

class MySafetyGate(Node):
    def __init__(self) -> None:
        super().__init__("my_safety_gate")
        self.declare_parameter("input_cmd_topic", "/cmd_vel_manual")
        self.declare_parameter("output_cmd_topic", "/cmd_vel")
        self.declare_parameter("front_stop_dist", 0.35)
        self._last_scan: LaserScan | None = None

        self.create_subscription(LaserScan, "/scan", self._scan_cb, 10)
        self.create_subscription(
            Twist, str(self.get_parameter("input_cmd_topic").value), self._cmd_cb, 10
        )
        self.cmd_pub = self.create_publisher(
            Twist, str(self.get_parameter("output_cmd_topic").value), 10
        )
        self.get_logger().info("my_safety_gate 就绪")

    def _scan_cb(self, msg: LaserScan) -> None:
        self._last_scan = msg # 只缓存，不发布

```

```

def _cmd_cb(self, msg: Twist) -> None:
    if self._last_scan is None:    # 传感器失明：不放行任何平移
        self.get_logger().warn("还没收到激光，拦截指令", throttle_duration_sec=2.0)
        self.cmd_pub.publish(Twist())
        return
    scan = self._last_scan
    dist = front_min_distance(scan.ranges, scan.angle_min, scan.angle_increment)
    stop = float(self.get_parameter("front_stop_dist").value)
    gated = gate_forward(msg, dist, stop)
    if gated.linear.x != msg.linear.x:
        self.get_logger().warn(
            f"前方 {dist:.2f}m 有障碍，截断前进速度", throttle_duration_sec=1.0
        )
    self.cmd_pub.publish(gated)

def main(args=None) -> None:
    rclpy.init(args=args)
    node = MySafetyGate()
    try:
        rclpy.spin(node)
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()

```

检验点 1——纯函数先测，一台裸 Python 就够（这就是分层的好处）：

```

cd ~/mecamind_ros2/src/nav_rebuild
python3 -c "
import math
from nav_rebuild.my_safety_gate import front_min_distance
# 造一帧假激光：360 束，全部 10 米远，但正前方那束 0.2 米
ranges = [10.0] * 360
ranges[0] = 0.2
print(front_min_distance(ranges, -math.pi, math.pi * 2 / 360)) # 应输出 0.2?
"

```

等等——上面这个测试会输出 `10.0` 而不是 `0.2`。**想清楚为什么再往下读。**（答案：`angle_min = -pi` 时，下标 0 对应的是**正后方**，不是正前方。正前方 0 弧度对应下标 180。把 `ranges[180] = 0.2` 再试。这个坑在真实调试里极常见：激光数组下标和方向的对应关系永远要先算再信。）

检验点 2——接到真系统里，用键盘遥控实测：

```
# 终端 1: 只启动轻量仿真 (有 /scan、/odom, 底盘订阅 /cmd_vel)
ros2 launch mecamind_bringup lite_sim.launch.py

# 终端 2: 你的安全门 (输入 /cmd_vel_manual, 输出 /cmd_vel)
ros2 run nav_rebuild my_safety_gate

# 终端 3: 键盘遥控, 注意 remap——它发的是 /cmd_vel_manual, 必须经过你的门
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -r cmd_vel:=/cmd_vel_manual
```

按 `i` 一直往前开, 撞墙前机器人应该自己停下 (终端 2 打出"截断前进速度"), 但按 `,` (后退) 和 `j/l` (旋转) 仍然有效——被堵住也能自己退出来。

工程版比你多想到的四件事 (每件都值得想 30 秒"如果没有会怎样"):

工程版特性	防的是什么事
四扇区 (前/后/左/右), 侧方管横移 <code>linear.y</code>	麦轮会横移, 只看前方会侧着撞墙
warn/stop 两级阈值 + 线性衰减	只有一条停止线会导致临界处"撞墙-后退"抖振
激光超时看门狗 (<code>scan_stale</code> 急停)	雷达驱动崩了, 门却拿着旧数据继续放行
指令超时看门狗 (<code>cmd_timeout</code> 补发零速)	上游节点崩了, 底盘锁在最后一条速度上狂奔

对照工程版: `src/mecamind_tools/mecamind_tools/safety_layer.py`。重点看 `analyze_scan` (两级阈值 + 线性衰减) 和 `_command_watchdog` (为什么用 `time.monotonic()` 而不是 ROS 时间——仿真暂停时 `sim time` 停走, 但"雷达断流"的检测不该跟着停)。

第 6 章 文件 4: `mission_executor.py`——用 action 客户端指挥 Nav2

这是最难的一个文件, 分两版写: **v1 同步版** (20 行, 5 分钟见效) 和 **v2 异步版** (真正的工程形态)。两版对比本身就是本章最大的知识点。

6.1 v1: 用 BasicNavigator 的同步版

Nav2 官方提供了封装好的 `BasicNavigator`, 把 action 的所有异步细节藏起来, 代码退化成"发目标 → 等结果 → 下一个"的顺序流程。

新建 `src/nav_rebuild/nav_rebuild/my_executor_v1.py`:

```

import time

import rclpy
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult

from nav_rebuild.nav_utils import pose_stamped_from_dict

# 目标点表 (与 config/mecamind_named_goals.yaml 一致, map 坐标系)
GOALS = {
    "hall_entry": {"x": -2.4, "y": -1.8, "yaw": 0.0},
    "living_room": {"x": 0.0, "y": 0.4, "yaw": 0.0},
    "kitchen": {"x": -0.5, "y": -1.2, "yaw": 0.0},
    "bedroom": {"x": 2.5, "y": 0.6, "yaw": 0.0},
}

def main(args=None) -> None:
    rclpy.init(args=args)
    nav = BasicNavigator()

    # 阻塞等 Nav2 全部 lifecycle 节点 active (新旧版本 API 兼容写法)
    try:
        nav.waitUntilNav2Active()
    except TypeError:
        nav.waitUntilNav2Active(localizer="amcl")

    for name in ["kitchen", "living_room", "bedroom"]:
        goal = pose_stamped_from_dict(GOALS[name], nav.get_clock().now().to_msg())
        nav.get_logger().info(f"====> 前往 {name}")
        nav.goToPose(goal)  # 内部就是发 navigate_to_pose 的 action goal
        while not nav.isTaskComplete():  # 同步轮询直到结束
            time.sleep(1.0)
            feedback = nav.getFeedback()
            if feedback:
                nav.get_logger().info(f"    剩余 {feedback.distance_remaining:.2f} m")
            result = nav.getResult()
            nav.get_logger().info(f"<==== {name}: {'成功' if result == TaskResult.SUCCEEDED else result}")

    rclpy.shutdown()

if __name__ == "__main__":
    main()

```

检验点:

```

# 终端 1: 完整导航系统
ros2 launch mecamind_bringup navigation.launch.py backend:=lite use_gui:=false with_rviz:=true

# 终端 2: 等 RViz 出图后
cd ~/mecamind_ros2 && colcon build --packages-select nav_rebuild && source install/setup.bash
ros2 run nav_rebuild my_executor_v1

```

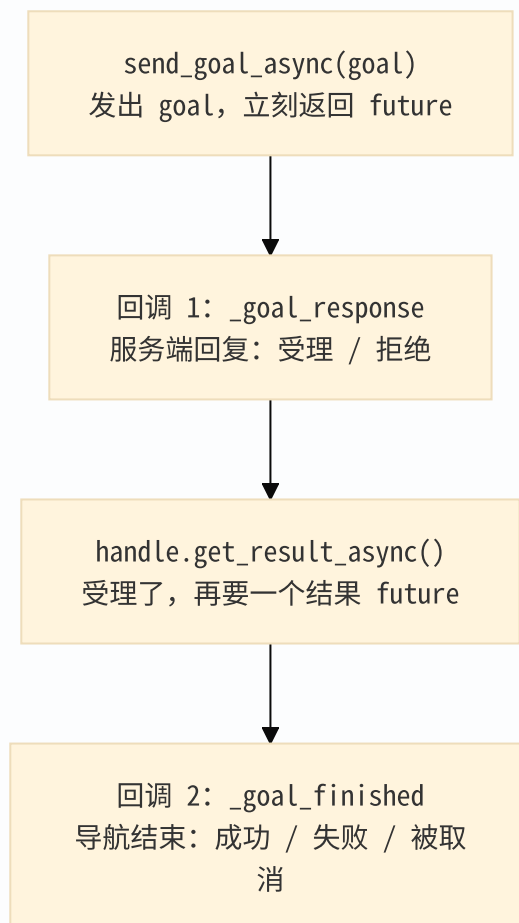
机器人应依次跑完三个房间。20 行代码完成三点巡航——爽，但试着回答：

v1 致命的问题：机器人去厨房的半路上，你想让它改去卧室（或者立刻停下），怎么办？——办不到。
`while not isTaskComplete()` 期间整个程序卡在循环里，没有任何机制接收新指令。**同步好写，但"聋"；要能随时听指挥，必须异步。**

6.2 v2: 自己写 ActionClient 的异步版

v2 的形态完全不同：它是一个**常驻节点**，永远在 `spin`，指令从 topic 进来，导航结果从回调回来，任何时刻都能被打断。

写之前先把 action 客户端的三环回调链背下来（实验 1.2 的三段输出，一一对应）：



新建 `src/nav_rebuild/nav_rebuild/my_executor_v2.py`：

```

import json

import rclpy
from action_msgs.msg import GoalStatus
from nav2_msgs.action import NavigateToPose
from rclpy.action import ActionClient
from rclpy.node import Node
from std_msgs.msg import String

from nav_rebuild.nav_utils import pose_stamped_from_dict
from nav_rebuild.my_executor_v1 import GOALS

class MyExecutor(Node):
    def __init__(self) -> None:
        super().__init__("my_executor")
        self._client = ActionClient(self, NavigateToPose, "navigate_to_pose")
        self._goal_handle = None  # 当前活跃的 goal 句柄, None 表示空闲

        self.create_subscription(String, "/my/task_command", self._task_cb, 10)
        self._state_pub = self.create_publisher(String, "/my/mission_state", 10)
        self.get_logger().info("my_executor 就绪, 等待 /my/task_command")

    def _report(self, text: str) -> None:
        """状态同时打日志 + 发 topic, 让外部工具 (和你自己) 随时能看到。"""
        self._state_pub.publish(String(data=text))
        self.get_logger().info(f"状态: {text}")

    # ---- 入口: 指令从 topic 进来 ----
    def _task_cb(self, msg: String) -> None:
        try:
            data = json.loads(msg.data)
        except json.JSONDecodeError as exc:
            self.get_logger().warn(f"指令不是合法 JSON: {exc}")  # 外部输入永远不可信
            return
        intent = str(data.get("intent", "")).strip().lower()

        if intent in {"stop", "cancel"}:
            if self._goal_handle is not None:
                self._goal_handle.cancel_goal_async()  # 异步请求取消
            self._report("canceled")
            return

        if intent != "navigate":
            self._report(f"unsupported:{intent}")
            return

        target = str(data.get("target", "")).strip()
        if target not in GOALS:
            self._report(f"unknown_goal:{target}")
            return

        if not self._client.wait_for_server(timeout_sec=3.0):
            self._report("nav2_not_ready")
            return

```



```

goal = NavigateToPose.Goal()
goal.pose = pose_stamped_from_dict(GOALS[target], self.get_clock().now().to_msg())
future = self._client.send_goal_async(goal)
# lambda 默认参数 name=target: 定义时刻就捕获当前值, 回调晚到也不会串
future.add_done_callback(lambda f, name=target: self._goal_response(f, name))
self._report(f"sending:{target}")

# ---- 回调 1: 受理响应 ----
def _goal_response(self, future, name: str) -> None:
    handle = future.result()
    if handle is None or not handle.accepted:
        self._report(f"rejected:{name}") # 服务端有权拒绝, 必须处理
        return
    self._goal_handle = handle
    self._report(f"active:{name}")
    handle.get_result_async().add_done_callback(
        lambda f, goal_name=name: self._goal_finished(f, goal_name)
    )

# ---- 回调 2: 最终结果 ----
def _goal_finished(self, future, name: str) -> None:
    self._goal_handle = None
    status = future.result().status
    if status == GoalStatus.STATUS_SUCCEEDED:
        self._report(f"reached:{name}")
    elif status == GoalStatus.STATUS_CANCELED:
        self._report(f"canceled:{name}")
    else:
        self._report(f"failed:{name}")

def main(args=None) -> None:
    rclpy.init(args=args)
    node = MyExecutor()
    try:
        rclpy.spin(node) # 常驻: 订阅回调和 action 回调都由 spin 驱动
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()

```

检验点 (导航系统保持运行):

```
# 终端 2: 你的执行器
ros2 run nav_rebuild my_executor_v2

# 终端 3: 盯着状态
ros2 topic echo /my/mission_state

# 终端 4: 发指令——去厨房
ros2 topic pub --once /my/task_command std_msgs/msg/String \
  '{data: "{\\"intent\\":\\"navigate\\",\\"target\\":\\"kitchen\\"}"}'

# 【关键实验】半路改主意：机器人还在去厨房的路上时，直接发"去卧室"
ros2 topic pub --once /my/task_command std_msgs/msg/String \
  '{data: "{\\"intent\\":\\"navigate\\",\\"target\\":\\"bedroom\\"}"}'

# 运动中取消
ros2 topic pub --once /my/task_command std_msgs/msg/String \
  '{data: "{\\"intent\\":\\"cancel\\"}"}'
```

半路改目标能生效（Nav2 的 `bt_navigator` 默认抢占旧 `goal`）——这就是 v1 做不到、v2 轻松做到的事。

6.3 v2 还差什么才是工程版？——三个"被时间打过的补丁"

工程版 `src/mecamind_tools/mecamind_tools/mission_executor.py` 比 v2 多的每一样东西，都对应一个真实发生过的时序 bug。读工程版时带着这三个问题去找答案：

1. **两阶段发送**（`_pending_send` + 0.5 秒定时器 `_tick`）：v2 里如果 Nav2 还没起来，`wait_for_server` 超时后指令就丢了。工程版把目标先"排队"，由定时器反复检查服务端就绪后才真正发送——指令永不丢失，最多显示 `waiting_for_nav2`。
2. **序列号防串扰**（`_goal_serial`）：v2 有个隐蔽 bug——旧 `goal` 刚被取消、它的 `result` 回调还在路上，此时新 `goal` 已经 `active`；旧回调到达后会状态错误地改成 `canceled:旧目标`。工程版每次发送/取消都给序列号 +1，回调回来先"验章"，对不上就直接返回。异步系统里，"迟到的回调"是万恶之源，序列号是最便宜的解药。
3. **超时兜底**（`_poll_active_goal`）：机器人被困住时 Nav2 可能反复重试、迟迟不返回结果。工程版记录 `goal` 受理时刻，超过 `goal_timeout_sec` 主动取消 + 发零速度急停。

外加两个业务扩展：`patrol` 模式（到点后在 `_goal_finished` 里把下一个路点重新排队，形成循环）和中文别名表（"厨房" → `kitchen`）。机制上没有新东西，读起来会很顺。

这三个补丁的代码真身、逐段精讲和状态字符串全集，见《10.3.3_工程版源码走读》第 5 章。

第 7 章 文件 5: launch——把所有零件拼成一台机器

现在所有零件都造好了，最后一步是"总装"。打开 `src/mecamind_bringup/launch/navigation.launch.py` 对照着读本章，它就是标准答案。

7.1 灵魂是 remapping：速度链是"改名字"改出来的

回忆实验 1.3 的三跳。它们在 launch 里的实现只有三处 remapping：

```
# ① controller_server 和 behavior_server: 默认发 cmd_vel, 改名成 /cmd_vel_raw
remappings=[("cmd_vel", "/cmd_vel_raw")]

# ② velocity_smoother: 输入接 /cmd_vel_raw, 输出改名 /cmd_vel_nav
remappings=[("cmd_vel", "/cmd_vel_raw"), ("cmd_vel_smoothed", "/cmd_vel_nav")]

# ③ safety_gate: 不用 remap, 它本来就是参数化的
{"input_cmd_topic": "/cmd_vel_nav"}, {"output_cmd_topic": "/cmd_vel"}
```

为什么 behavior_server 也要 remap? ——恢复行为（原地旋转、后退）也会直接发速度。漏掉它，机器人"自救"时的速度就绕过了平滑器和安全门，等于安全链上开了个后门。这类"每一个会发速度的节点都必须进链"的完整性检查，是集成层最容易翻车的地方。

7.2 启动顺序：为什么激活器要延迟 8 秒

launch 里所有 Node(...) 是同时启动的，但 Nav2 各节点从进程启动到 service 就绪要几秒（Gazebo 后端连 /clock 都要等好几秒）。所以：

```
lifecycle_activator = TimerAction(
    period=8.0,          # 等 8 秒再点火
    actions=[Node(package="mecamind_tools", executable="mecamind_lifecycle_activator", ...)],
)
```

而激活顺序本身写在节点清单里，顺序有讲究：

```
NAV2_LIFECYCLE_NODES = [
    "map_server",      # 先有地图
    "amcl",            # 才能定位（激活后立刻喂 /initialpose）
    "controller_server", # 然后才轮到规划/控制/行为树
    ...
]
```

7.3 双后端一个开关

backend:=gazebo|lite 的实现就是两个条件包含——Nav2 栈完全不用改，因为两个后端对外接口（/scan、/odom、/cmd_vel、TF）完全一致。接口统一，上层无感，这是整个项目最值得带走的架构经验。

7.4 练习：让你自己的三个节点上岗

终极练习——把官方 launch 里的三个自定义节点换成你写的：

1. 把 navigation.launch.py 复制到 src/nav_rebuild/launch/my_navigation.launch.py；
2. 把 mecamind_safety_gate / mecamind_mission_executor / mecamind_lifecycle_activator 三处的 package 和 executable 换成 nav_rebuild 和你的 my_safety_gate 等；

3. 参数名对不上的，照着自己的 `declare_parameter` 改：安全门是 `front_stop_dist` 等；激活器要把工程版的 `service_timeout_sec` 换成你的 `timeout_sec`，并把值调大到 60（9 个节点陆续上线需要时间，默认 20 秒在 Gazebo 后端可能不够）；
4. 你的 v2 执行器没有 `named_goals_file`、`waypoints_file` 等参数（目标表写死在代码里），删掉对应参数即可；
5. `setup.py` 的 `data_files` 里加上 `launch` 目录的安装（照抄 `mecamind_bringup` 的写法），构建后：

```
ros2 launch nav_rebuild my_navigation.launch.py backend:=lite use_gui:=false
```

能用你的指令话题 `/my/task_command` 把机器人指挥到三个房间，这套系统就是**你**实现的了。

注意：你的简化版激活器没有“发布初始位姿”功能（第 4 章进阶思考 1）。要么给它补上（照抄工程版 `publish_initial_pose`），要么启动后手动在 RViz 里用 **2D Pose Estimate** 在 origin 戳一下。推荐前者——这是本教程留的唯一必做作业。

第 8 章 验收：证明你的实现是真的能用

逐项打勾：

- ☐ 实验 1.1~1.3 的三个问题都能不看答案讲清楚；
- ☐ `my_activator` 能把手动启动的 `map_server` 点火到 `active`，返回码正确；
- ☐ `my_safety_gate` 在键盘遥控下能拦住撞墙，且倒车/旋转不受限；
- ☐ 检验点 1 那个“激光下标陷阱”能给同学讲明白为什么；
- ☐ `my_executor_v1` 顺序巡航三个房间成功；
- ☐ `my_executor_v2` 支持半途换目标和运动中取消；
- ☐ 能说出工程版 `mission_executor` 三个补丁（两阶段发送/序列号/超时）各防什么 bug；
- ☐ 第 7.4 的 `my_navigation.launch.py` 一条命令拉起你自己的集成层并完成导航；
- ☐ 最后跑一遍官方验收确认没把环境玩坏：

```
cd ~/mecamind_ros2 && ./scripts/test_cp3.sh
cat maps/mecamind_cp3_lite_nav_report.json
```

附：踩坑速查

症状	大概率原因	排查动作
<code>wait_for_service</code> 一直超时	目标节点没启动，或节点名拼错（少了 <code>/</code> 、名字不一致）	<code>ros2 service list grep 节点名</code>
目标一发就 <code>failed</code> ，日志有 "Initial robot pose is not available"	AMCL 没拿到初始位姿	RViz 2D Pose Estimate 戳一下原点；检查你的激活器是否发了 <code>/initialpose</code>
<code>ros2 topic pub</code> 发 JSON 没反应	引号转义错了	先 <code>ros2 topic echo /my/task_command</code> 看收到的原文是否合法 JSON
安全门不拦截	订阅的激光话题不对，或扇区角度算错	<code>ros2 topic info /scan</code> ；打印 <code>front_min_distance</code> 的返回值
机器人到点后状态一直不更新	<code>result</code> 回调没注册（漏了 <code>get_result_async().add_done_callback()</code> ）	对照 6.2 的三环回调链逐环打日志
半途换目标后状态显示成旧目标的结果	迟到的旧回调污染状态（v2 已知问题）	读工程版 <code>_goal_serial</code> ，给你的 v2 也加上
自己的 <code>launch</code> 起来后机器人乱跑/不动	某个会发速度的节点没进 <code>remap</code> 链	<code>ros2 topic info /cmd_vel</code> 看发布者是不是只有安全门

下一步

先读 `docs/10.3.3_工程版源码走读.md`：拿你刚写完的简化版对照工程版逐文件串讲，把本教程没覆盖的 `waypoint_patrol.py`（同步写法对照）、`mecamind_nav2_params.yaml`（参数导读）和 `nav_acceptance.py` + `test_cp3.sh`（验收链）一次补齐。

你现在拥有的这套"任务 `topic` 进、`action` 驱动 Nav2、状态 `topic` 出"的接口，就是后续课程的地基：第五课的视觉跟随会和 Nav2 抢 `/cmd_vel`（速度仲裁器插在你熟悉的那条链上），第六课的语音大模型发出的正是你已经处理过的 `{"intent": "navigate", "target": "..."}` 。集成层写通一次，后面每一课都是在这张图上加零件。